applied sciences

MDPI

Article

Quality of Service and Congestion Control in Software-Defined Networking Using Policy-Based Routing

Inayat Ali *, Seungwoo Hong and Taesik Cheung

Electronics and Telecommunications Research Institute (ETRI), Gajeong-ro 218, Yuseong-gu, Daejeon 34129, Republic of Korea; swhong@etri.re.kr (S.H.); cts@etri.re.kr (T.C.)
* Correspondence: inayat@etri.re.kr

Abstract: Managing queuing delays is crucial for maintaining Quality of Service (QoS) in real-time media communications. Customizing traditional routing protocols to meet specific QoS requirements—particularly in terms of minimizing delay and jitter for real-time media—can be both complex and time-intensive. Furthermore, these protocols often encounter challenges when adapted for vendor-specific hardware implementations. To address these issues, this paper leverages the programmable features of Software-Defined Networking (SDN) to simplify the process of achieving user-defined QoS, bypassing the limitations of traditional routing protocols. In this work, we propose a policy-based routing module that integrates with traditional routing protocols to ensure QoS for real-time media flows. QoS is achieved by rerouting the flow along a new low-latency path calculated by the proposed module when the queuing delay exceeds a certain threshold. The experimental results demonstrate that the proposed solution significantly enhances the performance of traditional routing protocols within an SDN framework, effectively reducing the average end-to-end delay by 80% and total packet loss by 73%, while also improving jitter and alleviating network congestion.

Keywords: QoS; congestion; SDN; policy-based routing; programmable networks

check for updates

Citation: Ali, I.; Hong, S.; Cheung, T. Quality of Service and Congestion Control in Software-Defined Networking Using Policy-Based Routing. *Appl. Sci.* **2024**, *14*, 9066. https://doi.org/10.3390/app14199066

Academic Editor: Alessandro Lo Schiavo

Received: 9 September 2024
Revised: 29 September 2024
Accepted: 30 September 2024
Published: 8 October 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Ensuring Quality of Service (QoS) is critical for delay-sensitive applications, especially in real-time communications like video conferencing and live media streaming. Among the performance metrics for such applications, delay is one of the most important, with queuing delay being a primary contributor to the end-to-end delay experienced by packets in networks [1,2]. Real-time applications like video conferencing have stringent requirements for both delay and QoS due to their interactive nature. Even minor increases in delay or packet loss can severely disrupt interactivity, resulting in a poor user experience. These performance issues are often caused by network congestion, which leads to excessive packet delays and losses, thereby degrading QoS.

To manage network congestion and ensure QoS, two main approaches are typically employed. The first approach is end-to-end congestion control, where transport protocols at the end nodes, such as TCP, QUIC [3], and RTCP, handle congestion while attempting to maintain QoS. The second approach involves controlling congestion and improving QoS at the network level by using techniques like Active Queue Management (AQM) [4]. While AQM marks or drops TCP packets when they are ECN-enabled [5], it does not distinguish between different flows and therefore lacks the ability to provide flow-specific QoS, treating all packets the same, except for ECN-enabled TCP packets. Moreover, real-time multimedia applications predominantly rely on UDP, which cannot tolerate the latency associated with packet retransmission in TCP, complicating the ability to maintain QoS in congested networks.

Since UDP is an unreliable protocol, real-time multimedia applications depend on the Real-Time Transport Protocol (RTP) [6] over UDP for media delivery. RTP is typically

used in conjunction with the RTP Control Protocol (RTCP), which provides feedback about the status of RTP flows to end hosts. Based on RTCP feedback, RTP adapts to changing network conditions, reducing the transmission rate when delays or packet losses are detected. However, this adaptation often results in reduced throughput, which negatively impacts media quality and overall QoS by lowering the bitrate.

Traditional routing protocols, such as RIP and OSPF, are still widely deployed and are unlikely to be replaced in the near future. However, these protocols do not provide QoS for specific network flows, as they treat all packets equally, regardless of their QoS requirements. Modifying these protocols to support QoS is a complex and labor-intensive task, and any changes may not be compatible with vendor-specific hardware. In contrast, Software-Defined Networking (SDN) offers programmability and flexibility, revolutionizing research and development in computer networks. Although writing complete routing protocols from scratch remains challenging, SDN enables network administrators and developers to write simple policy programs that automate network behavior, dynamically adjust routing decisions, and ensure QoS in real-time multimedia environments.

The increasing demand for flexible, customized QoS tailored to specific network flows, such as video conferencing, highlights the limitations of traditional networks. While traditional networks can offer basic QoS, they are typically constrained by standard protocols and vendor-specific configurations, limiting their ability to deliver user-defined QoS. The programmability of SDN provides an opportunity to overcome these challenges by enabling more flexible and customized QoS policies. Furthermore, modifying standard routing protocols to improve performance and QoS is a complex and resource-intensive process, often restricted by hardware compatibility issues. This motivated our exploration into how SDN's programmability can facilitate user-defined QoS, providing a more adaptable and efficient solution for real-time media applications.

In this work, we propose an SDN-based network that integrates traditional routing protocols, specifically configured using RouteFlow and the Quagga routing engine , to address these QoS challenges. We introduce a policy-based routing (PBR) module within the SDN controller that detects network congestion, adjusts routing dynamically, and ensures QoS by identifying alternative low-delay paths. The PBR module continuously monitors queuing delay, and when it exceeds a predefined threshold, it reroutes packets through alternative paths with lower delays to maintain high QoS standards. Our experimental results demonstrate that incorporating the PBR module with traditional routing protocols significantly reduces network congestion, improves key QoS metrics—including end-to-end delay, jitter, and packet loss—and ensures better overall performance for real-time multimedia applications.

In summary, the main contributions of this study are as follows:

- Development of an SDN-based network that integrates OSPF protocols, configured using RouteFlow and the Quagga routing engine in a virtualized environment.
- Mathematical modeling and approximation of the next network hop to determine the optimal data path for forwarding real-time multimedia.
- Implementation of the proposed policy-based routing for QoS in real-time multimedia and evaluation of its performance in a network emulator.

The remainder of this paper is organized as follows: Section 2 reviews related work. Section 3 provides an overview of SDN and RouteFlow. In Section 4, we present the proposed policy-based routing for QoS in real-time multimedia. The network environment used in our study is described in Section 5. The results are discussed in Section 6, and finally, we conclude this paper in Section 7.

2. Related Work

The flexibility and programmability of SDN have introduced new ways to address traditional network challenges, including congestion control and QoS requirements. SDN offers a global view of the network, enabling more informed decisions regarding congestion control and QoS. However, most networks rely on end-host-based congestion control,

which is not fully optimized for SDN because the SDN controller primarily interacts with forwarding devices and not the end hosts. To bridge this gap, several solutions have been proposed. For instance, OpenTCP, introduced in [7], installs lightweight agents on end hosts to modify TCP sessions dynamically based on the global network view provided by the SDN controller. This work has inspired various other solutions in which network-wide information is communicated to the sending host to adapt TCP for congestion alleviation [8–10]. However, these approaches require modifications to the existing SDN paradigm to allow the controller to communicate with end hosts. Moreover, these solutions focus on congestion control by reducing transmission rates, which benefits overall network performance but may not cater to specific flows.

A queue-length-based congestion control mechanism is proposed in [11,12], where the controller is informed of queue length status via OpenFlow protocols. When congestion is detected, the controller adjusts the TCP ACK receive window size for elephant flows in its flow table, prompting the sending host to reduce its transmission rate. Similarly, the authors in [13] add queue length information to the ACK packet, notifying the TCP sender to adjust its transmission rate accordingly. While these mechanisms reduce congestion and packet drops, they do not guarantee low-delay or stable jitter for real-time network flows.

In addition to host-based QoS and congestion control solutions, several researchers have explored adapting existing routing protocols for SDN environments, enabling new opportunities for customizing these protocols. In [14], the authors developed RouteFlow, an IP routing service for SDN that uses network function virtualization to deliver traditional IP-based routing services within SDN. The authors in [15] demonstrated the efficiency of using RouteFlow for routing services in SDN, showing its resilience to both single and multiple link failures. Their evaluation of OSPF within RouteFlow revealed that it provides shorter failover times compared to legacy OSPF implementations.

In [16], the authors proposed achieving near-zero switchover times by combining Multipath TCP (MPTCP) with SDN. The OpenFlow protocol, which supports meter tables for each flow entry, enables QoS and traffic engineering features like rate limiting, and facilitates faster switchover times than traditional IP routing. Similarly, [17] introduced Smart OSPF (S-OSPF) for congestion control, which manages congestion at edge nodes while maintaining traditional OSPF functionality at intermediate nodes. Although S-OSPF is a promising approach for SDN congestion control, it primarily addresses congestion at the network edges, leaving the potential for congestion between edge nodes unaddressed.

An Inter-Domain Management Layer (IML) is introduced in [18], which decouples routing and policy control, aligning next-generation SDN systems with traditional BGP autonomous systems (ASes). While IML focuses on policy and flow control between domains, it does not explicitly address QoS.

The existing work, as discussed above, ensures network performance for all flows without providing enhanced QoS for the flows of interest. Additionally, these solutions require the sending host to manage the congestion window based on network conditions communicated by the SDN controller and primarily address link failures by rerouting traffic along the available routes without determining a low-latency path. In contrast, our proposed approach does not involve the sending host to trigger any flow control. Furthermore, it does not simply reroute traffic along the available alternate routes in the event of a primary link failure; instead, it prioritizes real-time flows and reroutes them along a low-latency path calculated by the proposed method itself when a certain queue delay threshold is met.

3. Overview of SDN and RouteFlow

For many years, the pace of innovation in computer networks has lagged behind other communication technologies. This slow rate of innovation is primarily due to the closed nature of traditional networking systems, which depend on vendor-specific hardware and software. These proprietary systems not only stifle innovation but also increase management and operational costs. Traditional networks are composed of three logical

planes: the control plane, the data plane, and the management plane. In the traditional model, the control and data planes are tightly integrated, a concept often referred to as the "inside-the-box" paradigm. To reduce complexity and operational costs, researchers have sought to redesign network architecture, leading to the emergence of programmable networks [19].

Software-Defined Networking (SDN) marks a significant shift in networking by decoupling the control plane from the data plane. This separation has garnered significant attention from both industry and academia due to SDN's potential to simplify network management and enhance flexibility. In an SDN environment, the control plane, typically centralized in a controller, has a global view of the network and is responsible for its intelligence, while the data plane, composed of forwarding devices such as switches and routers, executes the controller's instructions. OpenFlow, a key protocol for SDN, standardizes communication between the controller and the data plane, offering a common programming interface for SDN networks [20].

Traditional routing protocols like OSPF, BGP, MPLS, and EIGRP are widely deployed and extensively tested. However, these protocols cannot be directly applied to SDN-based networks. Efforts have been made to adapt these protocols for SDN environments. One such effort is CPQD's development of RouteFlow [14], a framework that enables centralized IP routing services in SDN. RouteFlow consists of three main components: the RouteFlow Controller (RFproxy), the RouteFlow Server (RFServer), and a virtual network within the RFServer. The RFproxy discovers network topology, while the RFServer maintains a virtual machine (VM) for each OpenFlow switch in the physical network. Each VM mirrors the physical switch's network interfaces (NICs) and runs an open routing protocol stack, dynamically maintaining connectivity via software switches. Once the virtual environment is configured, the routing engine (such as Quagga) populates routing tables, which are then translated into flow entries and applied to the corresponding switches.

Several researchers have proposed similar protocols that run as applications on top of the SDN controller. However, as demonstrated in [15], utilizing RouteFlow services is more efficient. In this paper, we use RouteFlow to configure Open Shortest Path First, (OSPF) within an SDN environment. We selected OSPF due to its open-source nature and its ability to provide minimal failover times when configured through RouteFlow. To ensure QoS for specific network flows, we further customize the SDN controller (RFproxy) by adding a policy-based routing (PBR) module. The PBR module monitors congestion by tracking queuing delay, and when a predefined threshold is reached, it identifies alternative routes to reduce delay and packet loss. Our experimental results validate the effectiveness of this approach.

4. Proposed Methodology

This section explains the proposed methodology for ensuring QoS using policy-based routing.

4.1. Proposed Policy-Based Routing

A key factor in ensuring QoS is the use of differentiated services, which can be effectively implemented through policy-based routing (PBR). SDN, with its inherent flexibility, makes PBR implementation more straightforward, practical, and efficient by utilizing flow-based protocols rather than traditional packet/frame-based protocols.

OSPF, a widely used routing protocol, determines routes based on a configured cost associated with each link, typically related to bandwidth. However, OSPF does not dynamically account for current network conditions, such as link congestion. Cisco routers, for example, calculate the OSPF cost as follows:

$$\text{Link Cost} = \frac{\text{Reference bandwidth}}{\text{Link bandwidth}} \quad (1)$$

While this approach is efficient, it does not address real-time link congestion, which can negatively impact network performance.

In this work, we develop a PBR module within the SDN controller and integrate it into OSPF to mitigate link congestion and ensure QoS for traffic profiles specified by the network administrator, referred to as QoS flows. This approach allows administrators to select specific QoS flows for enhanced performance. The PBR module continuously monitors the queuing delay after a QoS flow is initiated. If the queuing delay exceeds a predefined threshold, the PBR module identifies an alternative low-delay path (detailed in Section 4.1.2) and reroutes the QoS flow along that path.

Our focus is on achieving QoS for real-time multimedia communication that uses the Real-Time Transport Protocol (RTP) for data transmission. The results, discussed in Section 6, show that integrating the PBR module with OSPF effectively ensures QoS and reduces network congestion. Figure 1 illustrates the operation of an SDN network, with shaded areas highlighting the proposed policy-based routing module.

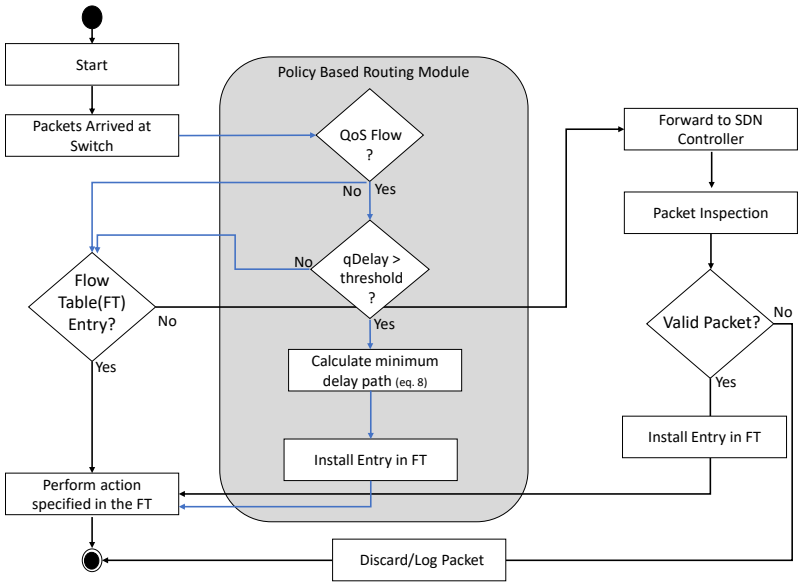


Figure 1. PBR module integration within SDN.

4.1.1. SDN-Based PBR Module

Software-Defined Networking (SDN) decouples the control plane from the data plane. The data plane consists of specialized forwarding hardware, such as switches, arranged in a specific topology and connected to the control plane via TCP. The control plane, managed by a logically centralized SDN controller with a global view of the network, makes all decisions regarding packet handling—whether to forward, drop, or log packets for future use.

As depicted in Figure 1, when a packet arrives at a switch, if it does not belong to an administrator-defined QoS flow and a corresponding flow table entry exists, the switch executes the action specified in the flow entry. This action may involve forwarding the packet to a specific port, sending it to the controller, modifying its fields, pushing/popping VLAN or MPLS headers, or dropping the packet. If no flow table entry exists, the packet is forwarded to the controller for further inspection. The controller checks the packet's validity; if the packet is invalid, it is discarded or logged. If valid, a flow table entry is created and sent to the switch via the OpenFlow protocol, and the action is applied to the entire flow.

For packets from a QoS flow, the PBR module is invoked. The PBR module first checks the current queuing delay at the switch where the packets from the QoS flow are received. If the queuing delay is below the predefined threshold, the flow follows the standard OSPF path. However, if the queuing delay exceeds or equals the threshold, the PBR module calculates an alternate minimum delay path, as described in Section 4.1.2, and diverts

the traffic to this new path. In this study, the threshold is set to 100 ms. The value of 100 ms is arbitrary, as this threshold can be adjusted based on the specific requirements of real-time applications. To examine the effect of different threshold values on network performance, we have included results in the Results section for various threshold values. The PBR module then creates a flow entry to reroute the traffic along the new minimum delay path and pushes the flow entry to the switch's flow table. The flow entry creation process involves the following steps:

- The module identifies links in the Shortest Path First (SPF) graph that have the lowest OSPF cost to the destination.
- The module checks the bandwidth and available buffer capacity of these links to select the optimal minimum delay path, as detailed in the next subsection.

Based on these parameters, the optimal route is selected, and the flow is forwarded along the best low-delay path. If congestion is detected on the OSPF route, the process is repeated to reroute the flow. The results in Section 6 demonstrate the effectiveness of our QoS assurance and congestion control approach.

4.1.2. Minimum Delay Path Calculation

Upon receiving a packet of a QoS flow, if the current queuing delay is greater than the threshold (th_1), the proposed model finds an alternate open path that leads to minimum delay. In a network, the overall delay experienced on a link between two nodes can be broken down into several components:

1. Transmission Delay (T_i): This is the time required to push all the packet's bits onto the link. It depends on the packet size S and the link bandwidth α_i.

$$T_i = \frac{S}{\alpha_i} \tag{2}$$

   Here, S is the packet size in bits, and α_i is the link bandwidth in bits per second.
2. Propagation Delay (P_i): This is the time it takes for the signal to travel from the sender to the receiver, depending on the distance d_i between the nodes and the propagation speed v of the signal in the medium (e.g., the speed of light in a fiber optic cable).

$$P_i = \frac{d_i}{v} \tag{3}$$

   d_i is typically measured in meters, and v in meters per second.
3. Queuing Delay (Q_i): This is the time a packet spends waiting in the queue before it can be transmitted. Queuing delay depends on the packet buffer occupancy β_i and the link bandwidth α_i. This delay is critical in congestion scenarios.
4. Processing Delay: This is the time taken by network devices (routers, switches) to process the packet header, perform routing decisions, etc. It is usually small and constant, so it can be neglected in this model.

In many cases, especially in well-provisioned networks, transmission and propagation delays are relatively stable and do not vary significantly. However, queuing delay can vary considerably depending on network conditions, making it the most critical factor in determining the overall delay.

Queuing delay Q_i can be approximated as the time it takes for the data already in the queue to be transmitted:

$$Q_i = \frac{\beta_i}{\alpha_i} \tag{4}$$

- β_i is the current buffer occupancy for link i (in bits or packets).
- α_i is the bandwidth of link i (in bits per second or packets per second).

The intuition behind this formula is straightforward: the delay increases with the amount of data already in the queue (more data to be sent) and decreases with higher bandwidth (faster transmission).

The total delay D_i on a link i is a sum of all the relevant delays:

$$D_i = T_i + P_i + Q_i \tag{5}$$

However, as noted earlier, if transmission and propagation delays are consistent across links, the queuing delay dominates in determining the best path. Therefore, we approximate the following:

$$D_i \approx Q_i = \frac{\beta_i}{\alpha_i} \tag{6}$$

- **Selecting the Next Hop**

Given that N_1 is connected to n other nodes ($N_2, N_3, \ldots, N_n$), as shown in Figure 2, the objective is to find the next hop N_{next} that minimizes the delay D_i. Mathematically, this is an optimization problem:

$$N_{\text{next}} = \arg \min_{i \in \{1,2,\ldots,n\}} \left(\frac{\beta_i}{\alpha_i} \right) \tag{7}$$

This means that the next hop is chosen based on the link with the smallest $\frac{\beta_i}{\alpha_i}$ ratio, indicating the least queuing delay and thus the least overall delay.

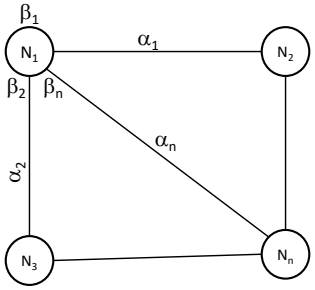


Figure 2. Optimal next hop selection.

The idea of using the smallest $\frac{\beta_i}{\alpha_i}$ ratio comes from queuing theory, particularly in models like the M/M/1 queue [21], where the service rate (bandwidth) and queue length determine the wait time. By choosing the path with the least queuing delay, the system ensures that packets are transmitted through less congested routes, improving overall network performance. The model is most applicable in stable network environments where transmission and propagation delays remain relatively constant, making queuing delay the primary factor. It is particularly effective in moderate to heavy traffic scenarios, where minimizing the queuing delay becomes crucial, as differences in delay are negligible in light traffic. The model assumes well-provisioned networks with consistent delays, such as modern high-bandwidth systems. In highly dynamic networks (e.g., wireless or mobile systems), additional factors like link reliability or adaptive algorithms may be needed to enhance decision making.

- Heavy Traffic: If the network is heavily loaded (high β_i), the queuing delay becomes significant, and the choice of N_{next} is crucial to avoid bottlenecks.
- Light Traffic: In lightly loaded networks (low β_i), all links might have similar delays, so the choice of the next hop could depend on other factors like load balancing or path reliability.

This model assumes that minimizing the immediate delay is the best strategy. However, in some scenarios, a slight increase in delay on one link could be justified by the

avoidance of future congestion or better overall network efficiency. However, for simplicity, the focus here is on immediate delay minimization.

To summarize, the mathematical model for choosing the next hop from N_1 is

$$N_{\text{next}} = \begin{cases} N_2 & \text{if } \frac{\beta_1}{\alpha_1} \leq \min\left(\frac{\beta_2}{\alpha_2}, \frac{\beta_3}{\alpha_3}, \ldots, \frac{\beta_n}{\alpha_n}\right) \\ N_3 & \text{if } \frac{\beta_2}{\alpha_2} \leq \min\left(\frac{\beta_1}{\alpha_1}, \frac{\beta_3}{\alpha_3}, \ldots, \frac{\beta_n}{\alpha_n}\right) \\ \vdots & \vdots \\ N_n & \text{if } \frac{\beta_n}{\alpha_n} \leq \min\left(\frac{\beta_1}{\alpha_1}, \frac{\beta_2}{\alpha_2}, \ldots, \frac{\beta_{n-1}}{\alpha_{n-1}}\right) \end{cases} \tag{8}$$

This generalized model effectively determines the optimal next hop from N_1 by focusing on minimizing the queuing delay, which is the most variable and critical component of network delay in many scenarios. By using the ratio $\frac{\beta_i}{\alpha_i}$, the model simplifies the decision-making process, allowing for efficient routing decisions that enhance network performance, particularly in congested environments.

In practice, this model can be expanded or adapted to include other network conditions, such as link reliability, error rates, or adaptive load balancing strategies, depending on the specific requirements of the network.

5. Network Environment

We conducted our simulations using the open-source Mininet (v. 2.2.0) network emulator [22] to construct a network topology consisting of Open vSwitches (OvSs), which communicate with an SDN controller (POX) via the OpenFlow protocol [20]. We utilized RouteFlow [14], an open-source project that provides virtualized IP routing services over OpenFlow-enabled routers and switches. A typical RouteFlow setup consists of an OpenFlow controller application (RFProxy), a standalone RouteFlow server (RFServer), and a virtual network environment that mirrors the physical network's connectivity while running IP routing engines like Quagga, as illustrated in Figure 3.

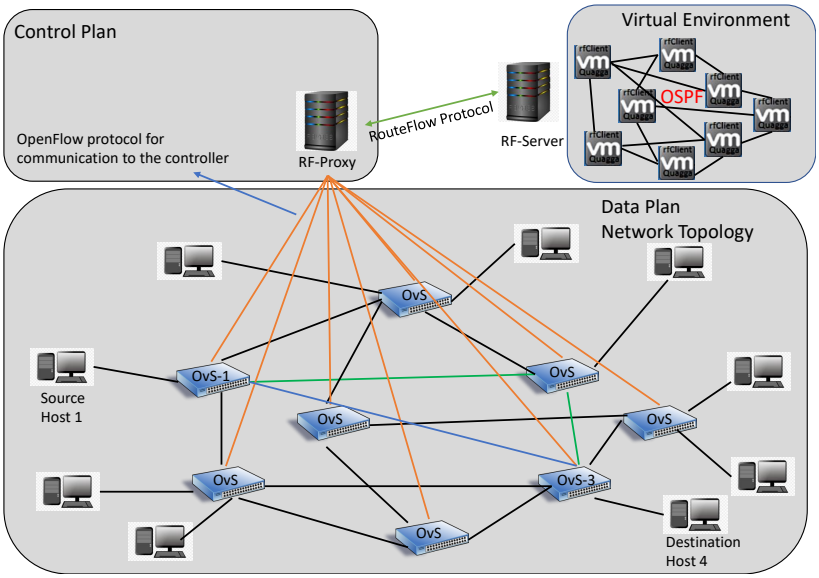


Figure 3. Network environment.

These routing engines create the forwarding information base (FIB) within the Linux routing tables, following rules defined by routing protocols such as OSPF and BGP. RouteFlow clients (RFClients) operate within the virtual network environment on the RFServer, collecting Linux IP and ARP table data and converting them into OpenFlow tuples. These tuples are then deployed to OpenFlow-compatible devices in the forwarding plane.

The entire setup is configured on a Linux guest system running within KVM on a Linux host. The network topology comprises eight Open vSwitches in the data plane, configured in a partial mesh topology, as shown in Figure 3. Each OvS in the data plane is paired with a virtual machine (VM) in the virtual plane, establishing a 1:1 mapping between each OvS and its corresponding VM. The VMs are equipped with the Quagga routing engine and a RouteFlow client.

The data plane switches communicate with the controller using the OpenFlow protocol, regularly reporting their status. This setup allows the controller to maintain a global view of the network topology. The RFProxy communicates this topology to the RFServer, ensuring that the VMs in the virtual plane are connected in a way that mirrors the physical topology of the data plane switches. Consequently, the virtual plane serves as an accurate representation of the data plane.

We configured the OSPF protocol on each VM using Quagga, populating the VMs' IP and ARP tables with OSPF routes. The RFClient, running as a daemon on each VM, collects these entries from the IP and ARP tables and forwards them to the RFProxy. The RFProxy then translates the OSPF routes into flow entries, which are installed into the corresponding switch flow tables via the OpenFlow protocol. This process repeats at regular intervals, enabling each switch in the data plane to function as an OSPF router and forward packets accordingly.

6. Results and Discussion

We evaluated the performance of the OSPF routing protocol on an SDN network, both with and without the policy-based routing (PBR) module, under high network congestion. The performance metrics assessed were packet loss percentage, end-to-end delay, jitter, flow completion time, and throughput.

In the experiment, Host 1 transmits real-time media (RTP) packets (QoS flow) to Host 4, as depicted in Figure 3. Additionally, each host sends UDP packets at a rate of 100 packets per second to random destinations at two-second intervals to create background traffic. The buffer size on each switch is set to 100 packets per port. OSPF selected the blue link, as shown in Figure 3, to forward the RTP packets to Host 4.

After heavily loading the network with traffic, we observed that 10.45% of the packets from QoS flow were lost due to congestion at one of the switches along the network path. This packet loss occurred because of buffer overflow at OvS-1 and OvS-3, which is the shortest path between Host 1 and Host 4, as calculated by the OSPF routing protocol. The packet loss percentage increased as more packets were transmitted, as illustrated in Figure 4. The increase in packet loss around 100 s indicates congestion at one of the intermediate switches, causing more packet drops. This observation is also verified by the throughput results in Figure 5, where we can see a significant drop in throughput at around 100 s.

However, with the PBR module enabled, the packet loss rate of QoS flow dropped significantly to 2.3%, with only a few packets being lost. This improvement was due to the PBR module identifying an alternate open route for the flow, represented by the green links between Host 1 and Host 4. Although the alternate route calculated by the PBR module using Equation (8) has more hops than the OSPF route, it experiences less congestion, resulting in lower delay. These results indicate a substantial performance improvement when OSPF is augmented with the PBR module. Figure 5 shows that the PBR module maintains higher and more stable throughput compared to OSPF under high congestion, owing to the PBR's ability to find alternative paths with lower end-to-end delay.

Jitter, an important network performance parameter for real-time communication, is significantly affected by congestion. As shown in Figure 6, the PBR module results in lower jitter compared to OSPF without the PBR module. The reason for this improvement is the reduction in packet loss, which stabilizes jitter, whereas OSPF without PBR suffers from higher variability due to packet drops.

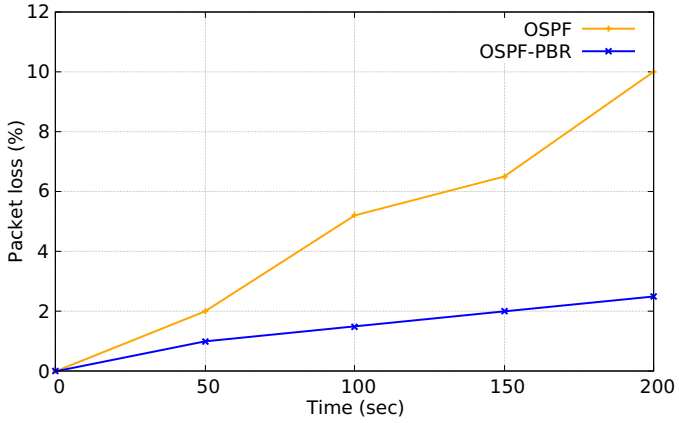


Figure 4. Packet loss comparison in OSPF and OSPF with PBR module.

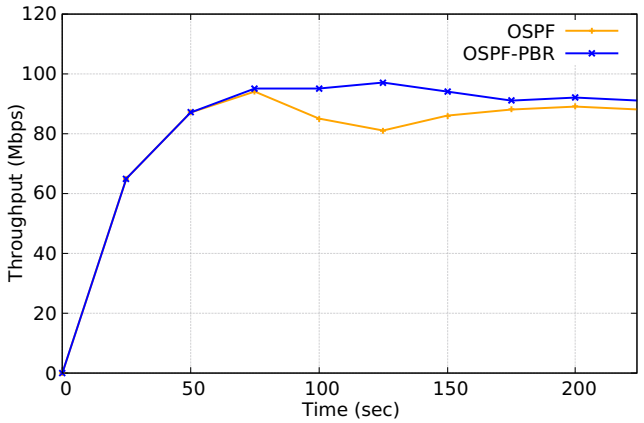


Figure 5. Throughput comparison in OSPF and OSPF with PBR module.

We also examined the effect of the PBR module on the flow completion time QoS flow. Figure 7 presents the flow completion times for OSPF with and without the PBR module for three different QoS flow sizes. The results show that adding the PBR module leads to shorter flow completion times. The improvement is less pronounced for smaller flows, as these flows do not cause significant congestion, resulting in only marginal gains from the PBR module.

End-to-end delay also improved with the PBR module. Without the PBR module, the end-to-end delay for QoS flow was approximately 1120 ms, as shown in Figure 8. The high end-to-end delay is primarily caused by queuing delays at intermediate switches due to buffer overflows. After incorporating the PBR module, the average delay was reduced to just 210 ms. As shown in Figure 8, when the queuing delay reaches a threshold (set by the administrator), the PBR module is triggered, finding an alternate low-delay path and diverting traffic accordingly, which results in a substantial reduction in end-to-end delay. In Figure 8, the queuing delay threshold is reached when 50 K packets are transmitted, and after that, the end-to-end delay drops significantly due to the alternate low-delay path selected by the PBR module. The PBR module's ability to dynamically find an alternate route for the RTP flow reduces buffer contention at intermediate switches, leading to this improvement.

Finally, we investigate the effect of different queue delay thresholds on network performance. As shown in Table 1, a high threshold value increases packet loss and end-to-end delay because, as the sojourn time of packets in the queue increases, it results in queue overflow and more packets being dropped. The increased sojourn time also reflects the average end-to-end delay. Packet loss and end-to-end delay decrease when the threshold is reduced from 150 ms to 100 ms and then to 50 ms. A lower queue delay threshold of 10 ms slightly increases flow completion time and end-to-end delay without significantly affecting packet loss. This is because such a low queuing delay in a high-load network

causes rapid switching of the QoS flow to a new low-latency route. This rapid switching adversely affects the QoS of the flow and increases end-to-end delay, as shown in Table 1.

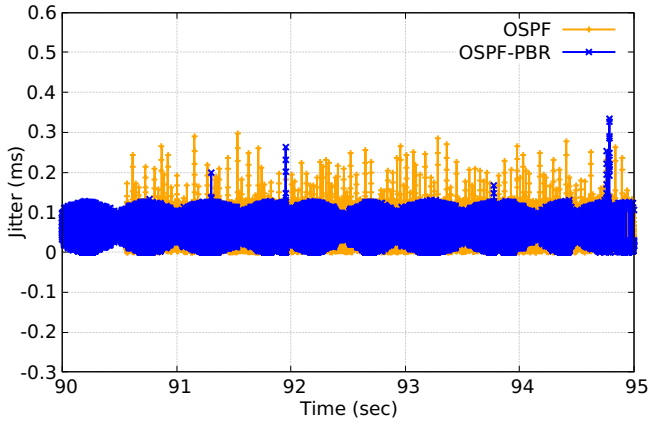


Figure 6. Jitter comparison in OSPF and OSPF with PBR module.

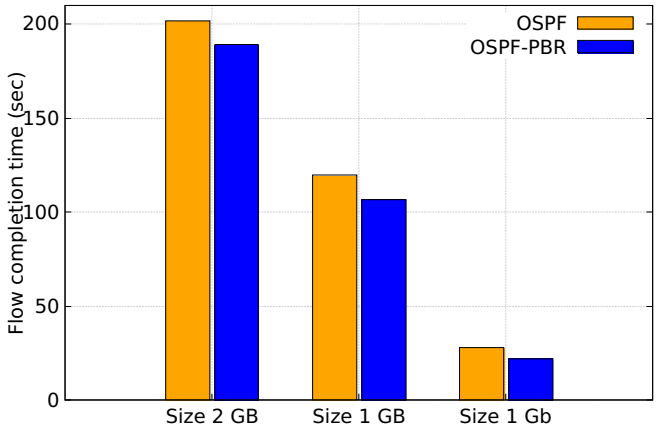


Figure 7. Flowcompletion time in OSPF and OSPF with PBR module.

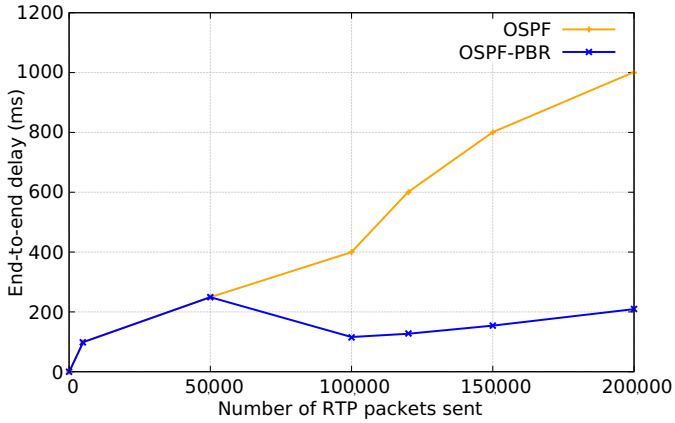


Figure 8. End-to-end delay comparison in OSPF and OSPF with PBR module.

Besides the significant performance gains discussed above, one potential limitation of this work is the absence of a mechanism that allows real-time applications to communicate their specific latency requirements directly to the SDN controller. This feature is important for enabling the controller to make more informed and dynamic routing decisions based on real-time application needs, which we plan to incorporate in future work.

Additionally, a future study could focus on further optimizing the PPBR module. For instance, integrating machine learning algorithms could significantly enhance the

system's ability to predict network congestion and make proactive routing adjustments, thereby improving QoS for real-time media flows. Machine learning techniques could also enable more adaptive and intelligent routing strategies, allowing the network to self-optimize based on historical traffic patterns. These improvements would extend the current framework's capabilities and offer more sophisticated, data-driven approaches to managing network performance.

Table 1. Effect of queue delay threshold on performance.

Queue Delay Threshold (ms)	Packet Loss (%)	E-to-E Delay (ms)	Flow Completion Time (sec)
150	5.1	235	112.7
100	2.3	210	110
50	2.1	198	109
10	2.1	257	115

7. Conclusions

In this paper, we proposed a policy-based routing (PBR) module integrated with OSPF in an SDN environment to address the challenges of managing QoS in real-time media communications. By leveraging SDN's programmability, the PBR module dynamically reroutes traffic based on real-time network conditions on a new minimum delay path, mitigating congestion and optimizing network performance. Experimental results confirm that the proposed module reduces end-to-end delay by 80%, decreases packet loss by 73%, and improves jitter, making it an effective approach to overcoming the limitations of traditional routing protocols. The PBR module's ability to detect congestion and find alternate low-delay paths was shown to effectively address the limitations of traditional OSPF routing, which does not account for real-time network conditions.

One potential limitation of this work is that it does not provide a mechanism for real-time applications to communicate their latency requirements to the SDN controller, which we plan to include in our future work. Our future work could explore further optimizations to the PBR module, such as integrating machine learning algorithms to predict congestion and enhance routing decisions.

Author Contributions: Conceptualization, I.A. and S.H.; methodology, I.A.; software, I.A.; validation, I.A., S.H. and T.C.; formal analysis, I.A.; investigation, I.A.; resources, S.H. and T.C.; data curation, I.A.; writing—original draft preparation, I.A.; writing—review and editing, I.A. and S.H.; visualization, I.A.; supervision, S.H. and T.C.; project administration, S.H.; funding acquisition, S.H. and T.C. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) [2021-0-00715, Development of End-to-End Ultra-high Precision Network Technologies].

Data Availability Statement: Data is contained within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

SDN	Software-Defined Network
OvS	Open vSwitch
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
QoS	Quality of Service
RTP	Real-Time Transport Protocol

RTCP	RTP Control Protocol
OSPF	Open Shortest Path First
RIP	Routing Information Protocol

References

1. Ye, J.; Leung, K.C.; Low, S.H. Combating bufferbloat in multi-bottleneck networks: Theory and algorithms. *IEEE/Acm Trans. Netw.* **2021**, *29*, 1477–1493. [CrossRef]
2. Carlucci, G.; De Cicco, L.; Mascolo, S. Controlling queuing delays for real-time communication: The interplay of E2E and AQM algorithms. *ACM Sigcomm Comput. Commun. Rev.* **2018**, *46*, 1–7. [CrossRef]
3. Ali, I.; Hong, S.; Park, P.K.; Kim, T.Y. Performance Evaluation of Transport Protocols and Roadmap to a High-Performance Transport Design for Immersive Applications. In Proceedings of the 2023 Fourteenth International Conference on Ubiquitous and Future Networks (ICUFN), Paris, France, 4–7 July 2023; pp. 926–931.
4. El Fezazi, N.; Elfakir, Y.; Bender, F.A.; Idrissi, S. AQM congestion controller for TCP/IP networks: Multiclass traffic. *J. Control. Autom. Electr. Syst.* **2020**, *31*, 948–958. [CrossRef]
5. Ali, I.; Hong, S.; Park, P.K.; Kim, T.Y. Rethinking Explicit Congestion Notification: A Multilevel Congestion Feedback Perspective. In Proceedings of the 34th Edition of the Workshop on Network and Operating System Support for Digital Audio and Video, Bari, Italy, 15–18 April 2024; pp. 64–70.
6. Sarker, Z.; Perkins, C.; Singh, V.; Ramalho, M. RTP control protocol (RTCP) feedback for congestion control. In *Internet RFC*; Internet Society: Geneva, Switzerland, 2021; (8888).
7. Ghobadi, M.; Yeganeh, S.H.; Ganjali, Y. Rethinking End-to-End Congestion Control in Software-Defined Networks. In Proceedings of the 11th ACM Workshop on Hot Topics in Networks, Redmond, WA, USA, 29–30 October 2012; pp. 61–66.
8. Hwang, J.; Yoo, J.; Lee, S.; Jin, H. Scalable Congestion Control Protocol Based on SDN in Data Center Networks. In Proceedings of the 2015 IEEE Global Communications Conference (GLOBECOM), San Diego, CA, USA, 6–10 December 2015; pp. 1–6.
9. Jouet, S.; Perkins, C.; Pezaros, D. OTCP: SDN-managed Congestion Control for Data Center Networks. In Proceedings of the NOMS 2016, IEEE/IFIP Network Operations and Management Symposium, Istanbul, Turkey, 25–29 April 2016; pp. 171–179.
10. Khan, A.Z.; Qazi, I.A. RecFlow: SDN-based Receiver-Driven Flow Scheduling in Datacenters. *Clust. Comput.* **2020**, *23*, 289–306. [CrossRef]
11. Lu, Y.; Zhu, S. SDN-based TCP Congestion Control in Data Center Networks. In Proceedings of the IEEE 34th International Performance Computing and Communications Conference (IPCCC), Nanjing, China, 14–16 December 2015; pp. 1–7.
12. Lu, Y.; Ling, Z.; Zhu, S.; Tang, L. SDTCP: Towards Datacenter TCP Congestion Control with SDN for IoT Applications. *Sensors* **2017**, *17*, 109. [CrossRef] [PubMed]
13. Lu, Y.; Fan, X.; Qian, L. EQF: An Explicit Queue-Length Feedback for TCP Congestion Control in Datacenter Networks. In Proceedings of the 2017 Fifth International Conference on Advanced Cloud and Big Data (CBD), Shanghai, China, 13–16 August 2017; pp. 69–74.
14. Sharma, S.; Colle, D.; Pickavet, M. Enabling fast failure recovery in openflow networks using routeflow. In Proceedings of the 2020 IEEE International Symposium on Local and Metropolitan Area Networks, Orlando, FL, USA, 13–15 July 2020; pp. 1–6.
15. Zeng, P.; Nguyen, K.; Shen, Y.; Yamada, S. On the Resilience of Software Defined Routing Platform. In Proceedings of the Asia-Pacific Network Operation and Management Symposium (APNOMS), Hsinchu, Taiwan, 17–19 September 2014.
16. Nguyen, K.; Minh, Q.T.; Yamada, S. Towards Optimal Disaster Recovery in Backbone Networks. In Proceedings of the IEEE 37th Annual Computer Software and Applications Conference, Washington, DC, USA, 22–26 July 2013.
17. Nakahoda, Y.; Naito, T.; Oki, E. Implementation of SmartOspf in Hybrid Software-Defined Network. In Proceedings of the 4th IEEE International Conference on Network Infrastructure and Digital Content 2014, Beijing, China, 19–21 September 2014; pp. 374–378.
18. Thai, P.; Oliveira, J. Decoupling Policy from Routing with Software Defined Interdomain Management. In Proceedings of the International Conference on Computer Communications and Networks (ICCCN), Nassau, Bahamas, 30 July–2 August 2013; pp. 1–6.
19. Kianpisheh, S.; Taleb, T. A survey on in-network computing: Programmable data plane and technology specific applications. *IEEE Commun. Surv. Tutor.* **2022**, *25*, 701–761. [CrossRef]
20. Miguel-Alonso, J. A research review of OpenFlow for datacenter networking. *IEEE Access* **2022**, *11*, 770–786. [CrossRef]
21. Moltafet, M.; Leinonen, M.; Codreanu, M. Average age of information for a multi-source M/M/1 queueing model with packet management. In Proceedings of the 2020 IEEE International Symposium on Information Theory (ISIT) 2020, Los Angeles, CA, USA, 21–26 June 2020; pp. 1765–1769.
22. Mininet Forum. Available online: https://mininet.org (accessed on 6 September 2024).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.